OOP principles for CUDA and GPU Programming

Stefan Caldararu

Abstract

The CUDA programming language has recently gained popularity as the primary language used for NVIDIA GPU programming. CUDA is a C-style language, and has only recently been extended to implement C++ style features and integrate with C++ projects. As such, low level memory management is still required - easily breaking C++ OOP principles. In this project, we explore methods of applying good memory management techniques to CUDA projects - specifically the principles Resource Allocation Is Initialization (RAII), the Rule of 3/5/0, and general abstraction techniques. Various GPU matrix multiplication examples are provided, demonstrating the implementations of these techniques.

Introduction

This project explores how Object-Oriented Programming (OOP) principles from C++ can be applied to CUDA programming. CUDA is the programming language used when writing code for NVIDIA GPUs. While CUDA was originally designed based on the C programming language, modern CUDA development has started to incorporated many higher-level features from C++. This allows developers to use object-oriented design ideas in GPU-accelerated programs. However, CUDA is still a fairly new language that is undergoing constant development. CUDA programming still has many low-level characteristics inherited from C, such as requiring explicit memory management and direct control over hardware execution. As a result, integrating OOP principles into CUDA code can be difficult. This requires usage of good memory managemnt, which are inherited from C++.

In this project, we present a basic framework for CUDA programming using OOP principles, allowing for safe memory management and program usage. This is primarily done by incorporating C++ OOP principles into basic CUDA program design. In the following sections, we discuss relevant background relating to C++ OOP memory management, and a basic CUDA programming overview. Following this, a literature review is presented, discussing some related works. Finally, related concepts are discussed, and code examples demonstrating these concepts are provided for a matrix multiplication application.

Background

C++ OOP Memory Management

Object-oriented programming in C++ allows users to structure programs using abstractions such as classes, encapsulation, and inheritance. These concepts are all exactly the same as in Java. One major difference between C++ and Java is resource management. Unlike Java which uses purely automatic garbage collection, C++ requires the developer to manage memory and other resources, while still having some basic automatic garbage collection. As a result, C++ has a few concepts that help ensure safe resource management, allowing for programmers using developed packages to rely mostly on automatic garbage collection and resource management. Two important principles used in C++ are Resource Acquisition Is Initialization (RAII) (Reference 2024) and the rule of three/five/zero (rul 2025).

RAII is a design pattern where all resources an object will use are tied to the lifetime of the object itself. For example, any dynamic memory that is required is allocated in the constructor, and freed in the destructor. This ensures that resources are automatically cleaned up when the object goes out of scope (when the destructor is called). This can be seen in a few of the provided examples.

Closely related is the rule of three/five/zero, which describes how classes that manage resources should implement general member methods. The original rule of three states that if a class defines any of the following three: a destructor, copy constructor, or copy assignment operator, it needs to define all three. This has additionally been extended to the rule of five, which includes definition of the move constructor and move assignment operator. The move constructor was only officially added to C++ with the release of C++11, and so this extension to the principle was only added on recently. The move constructor looks very similar to the copy constructor, but is primarily offered as a memory manamgement efficiency for objects which are about to be destroyed. Rather than performing a deep copy of all member data, The data references are directly copied over to the new object.

The rule of zero tells developers to design high-level classes so that explicit resource management is unnecessary. This allows resource management to be handled by as few classes as possible, helping with memory safety and allowing the compiler to generate these methods automatically.

In the case of these examples, any class that would use our Matrix Multiplication class should have no need to manually allocate dynamic memory. Any such allocations should be fully handled by a lower-level class.

These rules are particularly relevant when working with GPU memory, where direct low-level memory management might be required, so proper safety techniques are especially relevent.

CUDA Background

In addition to C++ basic design principles (which add onto general OOP principles), understanding the basic CUDA program design is essential. CUDA programs consist of host code running on the CPU and device code executed on the GPU. The CPU is responsible for allocating memory on the GPU, transferring data between the host and the device, launching GPU kernels, and retrieving results from GPU memory. GPU kernels are functions executed on the GPU. When launched, many threads launch in parallel, allowing for thousands of identical calculations to operate on different data. A basic code example demonstrating this is shown in .

CUDA extends C/C++ with special function qualifiers (NVIDIA Corporation 2025) that indicate where code will execute. The `__global__` qualifier is used for kernel functions that are launched from the host and executed on the GPU. These functions are called using CUDA's special triple-angle-bracket syntax, which specifies the number of thread blocks and threads per block. Functions marked with the `__device__` qualifier execute only on the GPU and can be called from other GPU functions or kernels. Conversely, functions marked with `__host__` execute on the CPU and behave like standard C++ functions. CUDA also allows functions to be marked with both `__host__` and `__device__`, enabling them to compile for execution on either the CPU or GPU. Functions running on the GPU have predefined variables indicating the thread ID that is running the code. This allows different threads to operate on different sections of memory, enabling this hyper-parallelism.

Within Nvidia GPU architecture, threads are defined within a series of blocks. Each block contains many threads that can cooperate with one another via fast shared memory, but are forced to synchronize their execution in a lockstep fashion. Blocks themselves are scheduled independently, allowing for multiple independent blocks to run in parallel. This means that code branching is very detremental within blocks; if a thread takes a different branches of an `if` statement than all the other threads, then all threads within the block must wait to re-synchronize.

These programming models introduce many design challenges. Due to the low-level nature of GPU programming, developers must manualally manage GPU memory - directly contradicting C++ memory management principles for good OOP design. This project explores how these concepts interact and how C++ programming techniques can be used to create CUDA programs that are both maintainable (through good OOP design) and efficient. By fully wrapping GPU memory allocation and deallocation in C++ classes, risk of memory leaks is significantly reduced. This, along with abstraction away from CUDA-specific implementations are the main contributions of this work.

Background and Literature Review

General-purpose computing on graphics processing units (GPGPU) is a growing field, which is accelerating conputation for highly-parallelizable tasks. NVIDIA GPUs are designed to execute on many threads in parallel, making them particularly effective for tasks such as simulation, machine learning, and numerical computation (such as Partial Differential Equation solvers (Gloster and O'Naraigh 2019)). NVIDIA's CUDA programming language extends the C and C++ languages, allowing developers to write highly-parallel code, which potentially offers imense speedups. It is worth noting some studies have shown that performance differences between CPU and GPU implementations depend strongly on internal algorithm and hardware optimization, and if an algorithm can be parallelized eaisly (Lee et al. 2010). Many use cases do not fit this criteria, so it is not a catch-all speedup tool.

CUDA became popularized through it's publication in literature in 2008 (Nickolls et al. 2008). This work describes the CUDA programming model (introduced in 2006), showing how applications are structured around kernels that execute on the GPU, while the CPU decides on data movement and kernel launches. This requires developers to explicitly manage device memory and data transfers between host and device. As a result, programming often resembles low-level systems programming, where resource management and hardware-specific knowledge are critical to achieving high performance (Kirk and mei W. Hwu 2007). This potentially breaks the high-level concepts of OOP.

Recent discussions in developer communities (Stack Overflow and Reddit) also highlight challenges of applying C++ design principles in CUDA programming. There are many trade-offs between abstraction and performance, as well as difficulties in safely managing GPU memory within object-oriented designs (Stack Overflow contributors 2022; Reddit r/CUDA contributors 2020). While abstraction should generally take precendence ("Human time"), it is also important to be aware of computational inefficiencies ("Computer time).

One approach to fixing this issue is Ikra-Cpp (Springer and Masuhara 2018), a domain-specific language in C++ that targets CUDA execution. Ikra-Cpp tries to enable object-oriented programming patterns for GPU applications while internally transforming data layouts and computation patterns to match GPU execution requirements. In particular, it uses a structure-of-arrays memory layout to improve memory access while still allowing developers to use higher-level abstractions. This work demonstrates that it is possible to use object-oriented design principles while maintaining good performance in GPU programming.

Concepts and Applicable Principles

Several C++ design principles are particularly useful when structuring CUDA programs. These include:

Encapsulation and Abstraction

Encapsulation allows CUDA-specific operations to be hidden behind simple interfaces (just like in Java). For example, a C++ class may internally manage GPU arrays and kernel launches while exposing only high-level operations such as matrix multiplication. There is no need for a user to know where their matrix multiplication is running. This approach reduces the complexity of high-level developer code.

RAII

RAII allows resources to be safely managed by tying them to the lifetime of a parent object. This generally applies to dynamic memory allocation. In CUDA programs, this concept can be applied directly to device memory allocation by wrapping GPU memory allocations inside classes. A class may allocate device memory in its constructor using `cudaMalloc` and release the memory in its destructor using `cudaFree`. This ensures that GPU memory is always released when the object goes out of scope.

Rule of Three/Five/Zero

When a class manages a resource such as GPU memory, the rule of three suggests that the class should implement a destructor, copy constructor, copy assignment operator, move constructor, and move assignment operator. These rules help prevent incorrect copying objects that manage GPU memory, which could potentially result in potential double frees or invalid memory access.

Project Example

To illustrate these concepts, a series of basic matrix multiplication examples is provided. These examples are described in the following subsections, and are provided. These examples are directly compilable on a computer with an Nvidia GPU along with the NVCC compiler, or runnable through the Google Colab notebook provided on a free-to-use T4 runtime (Google Research 2019). First, a standard CUDA matrix multiplication example is provided using none of the OOP principles discussed. Following this, an example abstraction fully wrapping all GPU code inside of a C++ class is provided, allowing the code to be called from an external cpp project. Additionally, a C++ class implementing proper RAII and rule of 3/5/0 techniques is implemented, demonstrating prevention of potential memory leaks in a more complicated project. Finally, an example is provided demonstrating further abstraction through the implementation of a Matrix interface, allowing for both CPU and GPU Matrix classes to be implemented. This final example fully abstracts the usage of GPU code from a package user, allowing them to benefit from the speedups of GPU computation without any knowledge outside of C++ standard programming.

Basic CUDA Example

In the provided Github repo, the `/basicGPU/` folder contains a basic GPU matrix multiplication example. The `matrixMul` kernel is a GPU-compilable function that gets run across a series of threads. Each thread first grabs it's block and thread ID, defining the data it will be operating on. This is launchable as a 3-dimensional ID. In this example, 2 of the dimensions are used, allowing for one dimension to define the row the thread will be operating on, and the other dimension to define the column. Once grabbed, a simple check is performed to ensure that the matrix component that will be operated on is not outside of the matrix bounds. This is an important check, as we rarely launch the exact number of threads as the size of the matrix. The number of threads launched is often dependent on our block size (the numeber of threads that operate at the same time, which is hardware-dependent). Following this, each thread iterates over the size (N) of our matrix, summing the product of the row-element and colum-elements. Finally, this summed result is inserted into the appropriate location in the result matrix.

This is a basic matrix multiplication example, and is in no ways optimal. Plenty of examples of matrix tiling (Wan 2023) for optimized matrix multiplication can be found online. Given that optimization is not the focus of this project, this kernel is deemed sufficient for this project and is used throughout the remainder of the examples.

Within the main function, we first initialize two large identity matrices for the computation. These are directly initialized as C-style arrays of floats. Following this, GPU memory is allocated for our two matrices, along with our result matrix.

GPU memory is allocated through a `cudaMalloc` call, which requires passing the address of a device pointer so that the routine can write back the location of the allocated memory on the GPU. This requires a double pointer, as the function must modify the original pointer variable itself rather than a local copy. By dereferencing this pointer-to-pointer internally, `cudaMalloc` assigns the newly allocated device memory address to the host pointer, enabling it to be used. Following this, the host-side A and B matrices are copied to GPU memory throught `cudaMemcpy` calls, which functions similarly to a standard c-style `memcpy` function call. The one addition to this is specification of the *direction* we want to copy memory. CUDA doesn't automatically detect the location (GPU memory or CPU memory) of our data, and so a `cudaMemcpyHostToDevice` flag must be passed. The result C matrix is intialized to all 0's through the `cudaMemset` function call.

Following this memory setup, kernel launch parameters defining the number of threads and number of blocks to launch must be initialized. A fairly standard 16x16 block size is defined, allowing for 256 threads to run in unison on one block. This is defined within the built-in CUDA `dim3` data type, which automatically initializes any unspecified data (in this example, the z-dimension) to 1. The number of blocks that are needed is then defined based on the size of our matrix by taking the ceiling of the size of our matrix dividied by our block size.

Finally, the matrix multiplication kernel is called with the defined number of blocks and threads per block. This is followed by a `cudaDeviceSynchronize` function call, which blocks our main CPU thread until all GPU threads have finished. This is followed by a memory copy back to host main memory, and a validation that our matrix multi-

plication yielded the expected results. All main memory and GPU memory is finally freed. This is a basic GPU code example, and is provided to give the reader an in-depth background on basic GPU computation.

GPU Class Example

In this example, a GPUMatrix class is defined, allowing for a user without understanding of how GPU code works to benefit from the computational efficiency. By wrapping all references to CUDA kernels within a class, a user can just include the standard C++ style header in their main method, without understanding of how the GPU impelmentation works.

In the `/classGPU/GPUMatrix.hpp` header file, a basic GPUMatrix class is defined. Two basic constructors are defined allowing for a user to initialize a matrix through just the size, or with a predefined matrix (as a double standard vector of floats). A destructor is also defined for automatic C++ garbage collection, and the copy constructors are disabled (these are included in the following RAII 3/5/0 example). A matmul function is defined, taking another GPUMatrix as an argument and directly returning the result as host memory. A `size` function is provided so the user can verify the size of the matrix.

In the CUDA implementation of this class, we first define the same kernel as in the last example. This cannot be included in the header file or in the class definition, as the header and class defintion must be purely C++ style code. Additionally, this helps ensure that the kernel is only called from within the CUDA implementation file.

Within the implementation of the GPUMatrix, the two constructors ensure GPU memory is allocated, and relevant data is copied over if provided. The destructor frees allocated memory when the object goes out of scope, helping ensure we have no memory leaks. In the matmul function, basic setup and multiplication occurs similar to what we see in the previous example. A major downside of this implementation is violation of the RAII principle, as we are allocating memory for our result matrix in the `GPUMatrix::matmul` function call. This is a simple example and so we can easily see that this is freed at the bottom of the function, but violates the RAII C++ OOP principle and so should be avoided. This is fixed in the following example.

RAII GPU example

The example found in `/RAIIGPU/` implements both RAII and the rule of 3/5/0 following good C++ OOP principles. This example is a direct extension of the previous one, and so only the changes made are covered in this section.

Within the header file for this example, we now add support for the copy constructor, copy operator, move constructor, and move operator. The copy constructor is a constructor that takes a reference to another `GPUMatrix` object, allowing it to directly copy all data from that object. The $=$ operator is defined so that a user can copy over `GPUMatrix` data with this operation. Similar definitions are made for the move constructor. Here, the major difference is in the method arguments. The move constructor takes an rvalue reference (denoted by `&&`), signaling to it that this is a temporary object that will soon go out of scope (likely at the end of this constructor call). This enables the move constructor to directly steal memory references from the temporary source object, avoiding inefficient deep copies of the underlying data (as with the copy constructor). This is standard C++ semantics, and can be read about in more detail (cppreference contributors 2026).

The implementations of these constructors nad operators can be found in `/RAIIGPU/GPUMatrix.cu`. The copy constructor and operator have fairly basic implementatios, where we allocate memory and copy device memory to the new allocated memory location. The move constructors also have straightforward implementations, where we are copying the *references* to the data to our own references. It is important to remove these references from the source object to ensure when that object gets deallocated, the memory doesn't get freed.

In addition to these implementations of the rule of 3/5/0, RAII principles are ensured to be followed. This primarily relates to the memory allocation within the `matmul` function call. This memory is now allocated and deallocated within the constructors and destructor, ensuring that it is all in one place, preventing memory leaks for larger scale projects.

Hiding GPU Example

The final example as a result of this work attempts to abstract the usage of a GPUMatrix from the user of a package. Here, a `Matrix` template is defined, which can be implemented by both a CPU implementation or GPU implementation. Implementations of this template must support multiplication with a *generic* matrix, which is primarily enabled through the `getData` methods. Within the CPU and GPU Matrix implementations, his is done by taking the data from the argument generic Matrix object and creating a matrix object with the same data. This data is stored in a Matrix implementation that matches the objects specific type, and then we call a private method that actually performs the multiplication. This helps enable cross-compatability between different Matrix implementations, where each implementation must handle multiplication by a generic Matrix on it's own.

Although the GPUMatrix and CPUMatrix classes are specifically named by their seperate types, These classes could potentially be implemented with the same header file, but different implementations of the individual methods. This would allow a project using this package to not depend on the implementation of the matrix multiplication library, allowing this to be determined at compile time dependent on system architecture.

Conclusion

CUDA programming provides powerful tools for accelerating highly parallel computations on GPUs. However, the low-level nature of CUDA memory management and kernel execution can make programs difficult to structure using traditional object-oriented design principles.

This project explores how C++ programming techniques

such as RAII, the rule of three/five/zero, and encapsulation can be used to improve the safety and maintainability of CUDA programs. By examining a matrix multiplication example implemented using C++ abstractions, this project aims to demonstrate practical ways to integrate object-oriented programming with GPU development.

Following a background on C++ programming and basic CUDA programming, a brief literature review of related works is provided. This is followed by applicable principles used in this project, and a series of examples that demonstrate usage of these principles in CUDA programming. These examples include a basic CUDA program to familiarize the reader with basic CUDA programming. Following this, the matrix multiplication kernel is first wrapped in a class, and then principles such as RAII and Rule of 3/5/0 are applied. Finally, an example fully abstracting the GPU Matrix behind a template.

Future work may explore additional frameworks, such as Ikra-Cpp, that attempt to provide higher-level abstractions for CUDA programming while maintaining efficient GPU execution. Additionally, a few examples are discussed but do not have implementations provided. Specifically, implementation of a compile-time change in CPU vs GPU Matrix multiplication implementations would be very interesting to explore. Finally, it would be interesting to explore any loss of Computer-Time due to these abstractions. Within the multiplication of the fully abstracted GPU Matrix, a new matrix must be created each time we want to perform a multiply. Running tests to see how much time this adds to the computation, and added memory usage could yield interesting results.

References

2025. The rule of three/five/zero. https://en.cppreference.com/w/cpp/language/rule_of_three.html. Accessed: 2026-02-23.

cppreference contributors. 2026. Move constructors. https://en.cppreference.com/cpp/language/move_constructor. Accessed: 2026-04-23.

Gloster, A.; and O'Naraigh, L. 2019. cuSten: A CUDA finite difference and stencil library. *SoftwareX*, 9: 184–189.

Google Research. 2019. Google Colaboratory. https://colab.research.google.com. Accessed: 2026-04-23.

Kirk, D. B.; and mei W. Hwu, W. 2007. NVIDIA CUDA Software and GPU Parallel Computing Architecture. In *Proceedings of the 6th International Symposium on Memory Management*, 103–104. ACM.

Lee, V. W.; Kim, C.; Chhugani, J.; Deisher, M.; et al. 2010. Debunking the 100x GPU vs. CPU Myth: An Evaluation of Throughput Computing on CPU and GPU. *ACM SIGARCH Computer Architecture News*, 38(3): 451–460.

Nickolls, J.; Buck, I.; Garland, M.; and Skadron, K. 2008. Scalable Parallel Programming with CUDA. *Queue*, 6(2): 40–53.

NVIDIA Corporation. 2025. *CUDA C++ Programming Guide: C/C++ Language Support*. NVIDIA. Accessed: 2026-02-23.

Reddit r/CUDA contributors. 2020. Learning about CUDA/GPU programming best practices. Accessed: 2026-02-23.

Reference, C. 2024. RAII — Resource Acquisition Is Initialization. Technical report, cppreference.com. Accessed: 2026-02-23.

Springer, M.; and Masuhara, H. 2018. Ikra-Cpp: A C++/CUDA DSL for Object-Oriented Programming with Structure-of-Arrays Layout. In *Proceedings of the Workshop on Programming Models for SIMD/Vector Processing (WPMVP'18)*, WPMVP, 1–9. ACM, New York, NY, USA; DOI:10.1145/3178433.3178439.

Stack Overflow contributors. 2022. Can you do C++-style CUDA? Accessed: 2026-02-23.

Wan, A. 2023. How to Tile Matrix Multiplication. https://alvinwan.com/how-to-tile-matrix-multiplication/. Accessed: 2026-04-23.